

به نام خداوند جان و خرد

پیش‌گزارش آزمایش دهم طراحی سیستم‌های دیجیتال

اعضای گروه

پوریا رحمانی ۴۰۲۱۱۱۴۱۸

امیرمحمد شربتی ۴۰۲۱۰۶۱۱۲

هدف آزمایش

در این آزمایش، باید یک پردازنده کوچک با دستورات محدود طراحی کنیم که از یک stack به جای registerfile استفاده می‌کند و نتایج میانی در آن ذخیره می‌شوند و I/O هم به صورت Memory_Mapped_IO است و سپس یک کد ماشین بنویسیم که پردازنده بتواند با آن یک ورودی X را دریافت کرده و حاصل $2*(X+23)-12$ را محاسبه کند و سپس حاصل را روی سون‌سگمنت نمایش دهیم.

فرضیات و مقادیر عددی

- دستورات ما ۱۲ بیتی هستند، ۴ بیت برای OpCode و ۸ بیت برای Operand که عدد ثابت یا آدرس حافظه است.
- PC ما یک رجیستر ۵ بیتی است.
- کدی که باید می‌نوشتیم را در یک ماژول که توضیح آن را در پایین می‌آوریم نوشتیم که مشابه یک ROM عمل می‌کند و PC قرار است روی آن حرکت کند.
- از ۸ خانه‌ای که برای I/O اختصاص دارد، آدرس F8 به ورودی کاربر اختصاص دارد.
- از آنجا که خروجی تابع هم باید بلافاصله روی FPGA نمایش داده شود، خانه FF را به مقدار ERROR (که یک است اگر با overflow یا منفی بودن ورودی روبرو شویم) و خانه FE را به عدد حاصل اختصاص دادیم که هر دو خانه از خانه‌های مربوط به I/O هستند.
- آدرس F0 که در سه تا از دستورات pop استفاده شده، صرفاً برای این است که POP نیاز به operand دارد و وقتی نیازی به آدرس اولین دستور مربوط به error handling را نیاز نداریم و می‌خواهیم آن را POP کنیم، آن را در این خانه از حافظه POP می‌کنیم و محتوای ارزشمندی ندارد!
- فرض بر این است که دستور sub عضو پایین top از پشته را منهای top می‌کند.
- از آنجا که مثلاً دستور popM باید در یک چرخه ساعت از پشته pop کند و در یک چرخه ساعت در حافظه بنویسد، همه کارها را در دو چرخه ساعت انجام می‌دهیم. در دستور push، pop، add و sub چرخه ساعت دوم به نوشتن error ها اختصاص دارد (همانطور که خواهیم دید، اگر پشته خالی باشد و دستور push اجرا شود، خروجی error خود را یک می‌کند)

- مدار یک خروجی error دارد (که آن را در IO می‌نویسد) که نشان می‌دهد از آخرین بار reset شدن تا کنون، آیا در عملیاتی error رخ داده یا خیر. بنابراین اگر خروجی error یک شود، به جز با reset کردن، به صفر برنمی‌گردد.
- در کد، فرض کردیم حالت $X=0$ هم غیر قابل قبول است و X باید از صفر بزرگتر اکید باشد.

ماژول‌های اصلی

Instmem ماژول

این ماژول یک ورودی 5 بیتی PC و یک خروجی 12 بیتی instruction دارد و مثل یک ROM عمل می‌کند و با case پیاده‌سازی شده که به ازای هر PC خاص یک دستور بر می‌گرداند. توضیح منطق کدی که در این ماژول نوشته شده، در بخش منطق کد آمده‌است. در تصویر زیر چندخط اول پیاده‌سازی آن را می‌بینید:

```
module Instmem (
    input [4:0] PC,
    output reg [11:0] instruction
);
always @(*) begin
    case (PC)
        5'd0: instruction <= 12'h1f8; //PUSHM F8 (X is stored in address F8 of memory)
        5'd1: instruction <= 12'h000; //PUSHC 0 (preparing to add X with zero to have sign of X in flag S)
        5'd2: instruction <= 12'h600; //ADD (now X is at the top of the stack and its sign is stored in S)
        5'd3: instruction <= 12'h01A; //PUSHC 26 (if error we should jump to error handling address)
        5'd4: instruction <= 12'h500; //JS (if X is negative, jump to address for error handling)
        5'd5: instruction <= 12'h400; //JZ (also, if X is zero, jump to address for error handling)
        5'd6: instruction <= 12'h2F0; //If X is positive, we don't jump, this will be executed and X will be at the top of the stack again
        5'd7: instruction <= 12'h017; //PUSHC 23
        5'd8: instruction <= 12'h600; //ADD
    endcase
end
```

AdvancedStack ماژول

این stack، از چهار عملیات push، pop، جمع و تفریق پشتیبانی می‌کند. برای همین، علاوه بر ورودی‌های معمول سیگنال‌های $rstN$ ، clk ، $active\ low$ ، $push$ ، pop و $data_in$ هشت بیتی، یک ورودی یک بیتی $is_addition$ نیز دارد. این ماژول، دو خروجی دارد. یکی خروجی error که زمانی که عملیات push باشد و پشته پر باشد، pop باشد و پشته خالی باشد یا جمع و تفریق باشد و پشته کمتر از دو باشد یک می‌شود. خروجی دیگر نیز $data_out$ است که در عملیات جمع و تفریق برابر سر پشته (حاصل عملیات) و در pop برابر داده بیرون انداخته شده است.

زمانی که push و pop هر دو صفر باشند، اتفاق خاصی نمی‌افتد. زمانی که push یک و pop صفر باشد، عملیات push و زمانی که pop یک و push صفر باشد، عملیات pop انجام می‌شود. پیاده‌سازی این دو، شبیه پیاده‌سازی stack معمولی در آزمایش چهارم است. زمانی که هر دو push و pop یک باشند، اگر $is_addition$ یک باشد، عملیات جمع اتفاق می‌افتد و زمانی که صفر باشد عملیات تفریق.

عملیات جمع به این صورت است که دو عنصر بالایی پشته جمع‌زده می‌شوند (به صورت ترکیبی با عملگر +) و به جای عنصر پایینی نوشته می‌شوند و pointer پشته یک واحد کم می‌شود و عملیات تفریق به این صورت است که عنصر بالایی پشته از عنصر پایینی کم می‌شود (به صورت ترکیبی با عملگر -) و به جای عنصر پایینی نوشته می‌شود، pointer پشته هم یک واحد کم می‌شود. (البته توجه کنید در هر دو صورت، اگر پشته کمتر از دو عضو داشت، خروجی error پشته یک می‌شود و اتفاق دیگری نمی‌افتد) تصویری از پیاده‌سازی این دو را در زیر می‌بینید:

```

else if(push && pop && pointer <= 1) error <= 1;
else if(push && pop && is_addition) begin
    stack_instance[pointer - 2] <= stack_instance[pointer - 1] + stack_instance[pointer - 2];
    pointer <= pointer - 1;
    full <= 1'd0;
    data_out <= stack_instance[pointer - 1] + stack_instance[pointer - 2];
end
else if(push && pop && !is_addition) begin
    stack_instance[pointer - 2] <= stack_instance[pointer - 2] - stack_instance[pointer - 1];
    pointer <= pointer - 1;
    full <= 1'd0;
    data_out <= stack_instance[pointer - 2] - stack_instance[pointer - 1];
end
end

```

ماژول Memory

این ماژول، شامل یک آرایه 256تایی از vectorهای هشت بیتی است. هشت خانه آخر آن به I/O اختصاص داده شده است. به طور خاص، در پیاده سازی خودمان خانه F8 را به ورودی کاربر، خانه FE را به خروجی تابع و FF را به خروجی error اختصاص دادیم (در کدی که در طرح آزمایش از ما خواسته شده، استفاده از همین سه خانه کفایت می کرد و بنابراین در مورد بقیه خانه های I/O تصمیمی نگرفتیم!) درست تر بود همه خانه هایی که به خروجی I/O اختصاص دادیم به عنوان خروجی این ماژول باشند، اما در این پیاده سازی، صرفاً همان ها که نیاز داشتیم، یعنی خانه های FE و FF را به عنوان خروجی مدار (علاوه بر خروجی read_data که خانه خوانده شده از حافظه است) را به عنوان خروجی در نظر گرفتیم.

ورودی های این مدار clk، rstN (ریست active low)، address (هشت بیتی)، data_in (هشت بیتی) و we (write enable) هستند.

read_data به طور مستقیم به خانه address از memory assign شده است (البته به شرط اینکه we صفر باشد، اگر we یک باشد، read_data، x خواهد بود. بنابراین برای خواندن از حافظه به clock نیاز نداریم. اما نوشتن در حافظه، با clk سنکرون شده است و در لبه مثبت چرخه ساعت، اگر we یک باشد، data_in را در خانه address از memory

```

module Memory (
    input [7:0] data_in,
    input clk,
    input we,
    input rstN,
    input [7:0] address,
    output [7:0] read_data,
    output [7:0] io_output,
    output [7:0] error_output
);
    reg [7:0] mem[255:0];
    assign io_output = mem[254];
    assign error_output = mem[255];
    assign read_data = we?8'bxxxxxxx:mem[address];
    always @(posedge clk, negedge rstN) begin
        if(!rstN) begin
            mem[255] <= 8'd0;
            mem[254] <= 8'd0;
        end
        else if(we) begin
            mem[address] <= data_in;
        end
    end
endmodule

```

می نویسیم.

ماژول EnRegister

همانطور که در ادامه خواهید دید، مدار را به صورت ساختاری پیاده‌سازی کرده‌ایم تا با آنچه در درس معماری کامپیوتر دیده‌ایم سازگارتر باشد و پیاده‌سازی برایمان راحت‌تر باشد. اما PC و رجیسترهای S و Z به enable نیاز دارند تا تنها در شرایط خاصی مقدارشان تغییر کند. بنابراین، برای آنکه درگیر پیاده‌سازی master-slave نشویم، یک ماژول رفتاری برای رجیستر enable دار نوشتیم که تنها در لبه مثبت چرخه ساعت، اگر enable یک بود، ورودی d را در خروجی q می‌ریزد:

```
module EnRegister #(
    parameter N = 8
)(
    input wire      clk,
    input wire      rstN,
    input wire      enable,
    input wire [N-1:0] d,
    output reg [N-1:0] q
);

    always @(posedge clk) begin
        if (!rstN)
            q <= {N{1'b0}};
        else if (enable)
            q <= d;
    end
endmodule
```

ماژول Cpu (مدار نهایی)

همانطور که گفتیم، این ماژول را به صورت ساختاری طراحی کردیم. ورودی‌های این مدار در لیست زیر آمده‌اند:

- Clk (ساعت)
- rstN (ریست active low)
- enable: درحالتی که صفر باشد، ورودی از کاربر گرفته می‌شود و در خانه I/O در نظر گرفته شده، یعنی خانه F8 نوشته می‌شود و اگر یک باشد، دستورات عادی CPU که در طرح آزمایش آمده‌است، اجرا می‌شوند.
- data_in: برای زمانی که enable یک باشد، ورودی‌ای است که کاربر برای نوشتن در بخش I/O حافظه می‌دهد.

این مدار دو خروجی error و io_output دارد که خروجی error بیت کم‌ارزش خانه FF حافظه‌ای است که در این ماژول تعبیه خواهیم کرد و io_output هم همان خروجی خانه FE آن است که به عنوان خروجی I/O از آن‌ها برای نمایش روی برد استفاده خواهیم کرد.

در این مدار از چهار EnRegister، یک AdvancedStack (به نام cpu_stack)، یک Instmem و یک memory (به نام cpu_memory)، instance گرفته شده است که نقش هر کدام را توضیح می‌دهیم و بیان می‌کنیم که هر کدام از ورودی‌های آن‌ها را به چیزی assign کرده‌ایم.

step: یک instance از EnRegister یک بیتی که enable آن به ورودی enable ماژول Cpu وصل است و d آن برابر نقیض q آن است. به این معنا که هر بار که در حال اجرای دستور باشیم (و کاربر در حال نوشتن ورودی خود نباشد) در هر چرخه ساعت، خروجی این رجیستر نقیض می‌شود. کاربرد این EnRegister تعیین این است که در کدام کلاک از دستور فعلی قرار داریم. نام خروجی این EnRegister، inst_step است.

S: یک EnRegister یک بیتی برای پرچم S. enable آن باید زمانی یک باشد که enable ماژول Cpu یک باشد، در یکی از دستورات add یا sub باشیم و عملیات جمع یا تفریق تمام شده باشد، یعنی در کلاک دوم دستور باشیم. یعنی $(opcode == 6 \mid \mid opcode == 7) \&\& inst_step \&\& enable$ و S_din باید برابر علامت خروجی stack که حاصل جمع یا تفریق را نشان می‌دهد باشد، یعنی $stack_dout[7]$

Z: یک EnRegister یک بیتی برای پرچم Z. enable آن دقیقاً مثل enable مربوط به S است و Z_din باید برابر صفر بودن $stack_dout$ که حاصل عملیات را نشان می‌دهد باشد. یعنی $stack_dout == 0$

pc: یک EnRegister پنج بیتی که نمایانگر pc است. استفاده از EnRegister برای pc به این دلیل است که خروجی pc باید در هر لبه کلاک که enable یک است و کلاک دوم دستور فعلی باشد عوض شود. بنابراین ورودی enable آن باید $inst_step \&\& enable$ باشد. (منظور از enable دوم، ورودی enable ماژول Cpu است). نام q آن pc_dout و نام d آن pc_din است. pc_din باید در زمانی که دستور JS است و S یک است (یعنی $S_dout == 5 \&\& opcode == 1$) یا JZ است و Z یک است (یعنی $Z_dout == 4 \&\& opcode == 1$) یا دستور Jump است (یعنی $opcode == 3$) برابر خروجی stack موجود در Cpu باشد که نام آن $stack_dout$ است. و گرنه مقدار آن باید $pc_dout + 1$ باشد.

Instruction_rom: یک instance از Instmem که ورودی آن pc_dout است و خروجی آن به یک wire به نام instruction وصل شده‌است. در نتیجه، همواره چهار بیت پرارزش instruction، opcode دستور مورد نظر را مشخص خواهد کرد و هشت بیت کم‌ارزش آن operand آن دستور را مشخص خواهد کرد.

cpu_stack: یک instance از AdvancedStack

- ورودی‌های clk و rstN آن از همان ورودی‌های ماژول Cpu آمده‌اند.
- ورودی push آن زمانی یک است که اولاً enable یک باشد و ثانیاً در کلاک اول دستور باشیم (یعنی inst_step صفر باشد) و در یکی از دستورات add، sub، pushC یا pushM باشیم. یعنی opcode بین 0,1,6,7 باشد.
- ورودی pop آن زمانی یک است که اولاً enable یک باشد و ثانیاً در کلاک اول دستور باشیم ($inst_step = 0$) و در یکی از دستورات add، sub، jump و popM باشیم (یعنی opcode یکی از 2,3,6,7 باشد) یا در دستور JZ باشیم و Z یک باشد ($Z_dout == 1 \&\& opcode == 4$) و یا در دستور JS باشیم و S یک باشد ($S_dout == 1 \&\& opcode == 5$)
- ورودی is_addition آن تنها زمانی یک است که در دستور add باشیم، یعنی $opcode == 6$
- ورودی data_in آن زمانی که در دستور pushc باشیم از operand دستور می‌آید (که برابر هشت بیت کم‌ارزش خروجی Instmem است) و در غیر این صورت از خروجی read_data مربوط به cpu_memory که instance ای از Memory است و در پایین در مورد آن توضیح خواهیم داد می‌آید (این مورد تنها در دستور pushM به کار می‌آید که ورودی از خروجی حافظه است)

cpu_memory: یک instance از Memory:

- ورودی‌های clk و rstN از همان ورودی‌های ماژول Cpu هستند.
- ورودی we در برقراری یکی از این سه شرط یک می‌شود: enable! به معنای این که کاربر در حافظه می‌نویسد، یا در یکی از دستورات add, sub, pushM, pushC یا pop باشیم و بیت error پشته یک باشد و در کلاک دوم دستور باشیم (یعنی `enable && (pc>5 || pc<3) && stack_error` inst_step یا اینکه در دستور pop باشیم، بیت stack_error صفر باشد و در کلاک دوم دستور باشیم یعنی `opcode == 2 && inst_step && !stack_error`)
- برای data_in در حالت اولی که برای نوشتن بیان شد، data_in حافظه باید همان ورودی data_in ماژول Cpu باشد. برای حالت دوم، مقدار نوشته شده باید 1 باشد و برای حالت سوم، مقدار نوشته شده، از stack_dout می‌آید.
- address در حالت enable! باید 0xF8 باشد یعنی محل ورودی دادن کاربر به I/O، در حالت نوشتن error که شروط آن ذکر شد، باید 0xFF باشد، یعنی محل در نظر گرفته شده برای نوشتن error در I/O و در غیر این صورت باید operand (هشت بیت کم ارزش خروجی Instmem) باشد که این حالت آخر برای دستورات pushM و pop به کار می‌آید.

ماژول testbench

در این ماژول یک instance از Cpu ساختیم و clk را هر 20 واحد زمانی not کردیم تا یک چرخه ساعت ایجاد شود. سپس از ساختار زیر برای بررسی صحت عملکرد کد استفاده کردیم و برای هر test از data_in متفاوتی استفاده کردیم تا بررسی کنیم مدار به درستی کد خواسته شده را برای مقادیر مختلف اجرا می‌کند. (البته عبارات print شده برای تست‌های مختلف قاعدتاً متفاوت بودند!!)

```
$display("\n-----Testing minimum possible invalid positive X-----");

rstN = 0;
enable = 1;
@(posedge clk);
#2;
enable = 0;
rstN = 1;
data_in = 8'd47;
@(posedge clk);
#2;
enable = 1;
#2500;
if(error == 1) begin
    $display("Passed, You turned the error bit on correctly!");
    passed = passed + 1;
end
else
    $display("Failed, 2*(X+23)-12 = %0d > 127 but error bit is off!", 2*(data_in+23)-12);
total = total + 1;
```

مقادیری که برای data_in تست کردیم به این صورت بودند:

ابتدا کوچکترین مقدار X که $2(X+23)-12$ از 127 بزرگتر باشد، یعنی 47 را تست کردیم که به درستی سیگنال error یک شد. سپس $X=46$ را تست کردیم که بزرگترین X قابل قبول است. سپس حالات $X=0$ و $X=-1$ را چک کردیم که در هر دو به درستی سیگنال error برابر یک شد. سپس پنج عدد تصادفی در بازه 1 تا 46 که بازه اعداد قابل قبول است تولید کردیم و

خروجی را بررسی کردیم که به درستی برابر $2(X+23)-12$ بود و error هم صفر بود. در نهایت، از آنجا که مطابق آنچه در پایین از منطق کد می خوانید، حالتی که $X+23$ هم از 127 بیشتر باشد، از مسیر متفاوتی زودتر به error می رسیم، یک عدد بررسی شد که $X+23$ بیشتر از 127 باشد که در آن هم به درستی error یک شد. در نهایت تعداد تست های پاس شده را چاپ کردیم که شکر خدا هر ده تست موفق بود! تصویر زیر مربوط به اجرای موفقیت آمیز این تست هاست:

```
[Running] tb.v
VCD info: dumpfile cpu.vcd opened for output.

-----Testing minimum possible invalid positive X-----
Passed, You turned the error bit on correctly!

-----Testing Maximum valid X-----
X = 46, 2*(X + 23) - 12 = 126, Your result = 126, Error output = 0
Passed!

-----Testing edge case with X = 0-----
Passed, X was not positive and you turned the error on!

-----Testing edge case with X = -1-----
Passed, X was negative and you turned the error on!

-----Testing some valid numbers-----
X = 27, 2*(X + 23) - 12 = 88, Your result = 88, Error output = 0
Passed!

X = 12, 2*(X + 23) - 12 = 58, Your result = 58, Error output = 0
Passed!

X = 1, 2*(X + 23) - 12 = 36, Your result = 36, Error output = 0
Passed!

X = 9, 2*(X + 23) - 12 = 52, Your result = 52, Error output = 0
Passed!

X = 25, 2*(X + 23) - 12 = 84, Your result = 84, Error output = 0
Passed!

-----Testing some number with X+23 > 127-----
Passed, X was so big and you turned the error on!

Passed 10 / 10 of tests
Passed all the tests!
tb.v:163: $finish called at 25602 (1s)
[Done] exit with code=0 in 0.864 seconds
```

فایل vcd شکل موج اجرای این تست بنچ نیز با نام cpu.vcd به پیوست ارسال شده است که می توانید مشاهده کنید.

منطق کدی که برای محاسبه $2(X+23)-12$ نوشتیم

نکته ای که در ابتدا به آن اشاره می کنیم این است که در صورت سوال گفته شده اگر حاصل بیش از حد مجاز (بیش از ۱۲۷) بود، باید error را یک کنیم. اما اگر ما ابتدا $2(X+23)-12$ را محاسبه کنیم و سپس ۱۲ واحد از آن کم کنیم، ممکن است $2(X+23)-12$ از ۱۲۷ بیشتر شود اما $2(X+23)-12$ از ۱۲۷ بیشتر نشود و در این صورت با اینکه طبق خواسته صورت آزمایش، خروجی Error نباید یک شود و باید حاصل درست را بدست آوریم، حاصل به دلیل overflow رخ داده، نادرست خواهد بود. بنابراین ما ابتدا $X+23$ را محاسبه کردیم (توجه کنید حاصل نهایی $2X+34$ است که طبق مثبت بودن X از $X+23$ بیشتر است، پس مقدار $X+23$ نباید overflow کند) و اگر دچار overflow شدیم (یعنی پرچم S برابر یک شد) به بخش error handling می پریم (با استفاده از دستور JS) و سپس دوازده واحد از آن کم می کنیم و بعد، یکبار دیگر $X+23$ را محاسبه می کنیم و با حاصل جمع می کنیم تا حاصل نهایی بدست آید. در نهایت، باز هم بررسی می کنیم اگر دچار overflow شدیم و پرچم S برابر یک شد، به آدرس مربوط به error handling پرش می کنیم (با استفاده از دستور JS)

برای بررسی کردن اینکه X مثبت است یا خیر، از آنجا که خواندن از حافظه به stack با PUSHM پرچمها را تغییر نمی‌دهد، یکبار صفر را به پشته PUSHC می‌کنیم و X را با صفر جمع می‌کنیم تا پرچمها تغییر کنند و همچنان X بالای پشته بماند. سپس، بررسی می‌کنیم اگر پرچم S یک بود (که یعنی X منفی است) یا اگر پرچم Z یک بود (که یعنی X صفر است، ولی در صورت سوال گفته شده X باید مثبت باشد) به بخش error handling می‌رویم.

رفتن به بخش error handling هم به این صورت است که PC مربوط به خط اول بخش error را در پشته PUSHC می‌کنیم و سپس با JS یا JZ به آن پرش می‌کنیم. بنابراین، در ادامه کد، اگر error ای وجود نداشت، باید ابتدا آدرس error که PUSH شده را POP کنیم و سپس به کار ادامه دهیم (چون در صورت آزمایش گفته شده که مثلاً در دستور JZ فقط زمانی که پرچم Z یک باشد، POP اتفاق می‌افتد و PC بروز رسانی می‌شود، یعنی در صورت صفر بود پرچم Z، POP هم نباید اتفاق بیفتد)

در بخش error handling نیز تنها کاری که می‌کنیم این است که مقدار 1 که مقدار ERROR است را در پشته PUSHC می‌کنیم و آن را با POP در آدرس FF که به مقدار ERROR اختصاص داده شده بود، ذخیره می‌کنیم.

در پایان کد هم دو خط نوشتیم که یک حلقه ایجاد کنند که از اضافه شدن مدام PC پس از اتمام کد جلوگیری شود تا PC overflow نکند و کد از اول اجرا نشود. این دو خط به این صورت هستند که در خط اول مقدار PC همان خط را به پشته PUSHC می‌کنیم و در خط بعدی JUMP را صدا می‌زنیم تا دوباره به خط بالا برگردد و این روند تا reset شدن PC ادامه پیدا کند.

نمایش روی seven segment

برای این کار یک ماژول کمکی به اسم CpuWrapper نوشتیم که یک Cpu درون خود دارد (بنابراین ورودی‌های clk، data_in، rstN (هشت بیتی) و enable دارد) و خروجی‌های آن یک بیت error و دو خروجی هفت‌بیتی hex0 و hex1 هستند که بیت‌های hex0 مربوط به قطعات seven segment اول هستند که چهار بیت کم‌ارزش io_output را در مبنای ۱۶ نشان می‌دهند و بیت‌های hex1 مربوط به قطعات seven segment دوم که چهار بیت پرازش io_output را در مبنای ۱۶ نشان می‌دهند. برای assign کردن بیت‌های hex0 و hex1 همان کاری را می‌کنیم که در تمرین‌های پیشین، مثل آزمایش ششم انجام دادیم، یعنی :

```
wire first_bits = io_output[3:0];
wire second_bits = io_output[7:4];

assign hex0[0] = (first_bits == 4'd1 || first_bits == 4'd4 || first_bits == 4'd11 || first_bits == 4'd13);
assign hex0[1] = (first_bits == 4'd5 || first_bits == 4'd6 || first_bits == 4'd11 || first_bits == 4'd14 || first_bits == 4'd15);
assign hex0[2] = (first_bits == 4'd2 || first_bits == 4'd12 || first_bits == 4'd14 || first_bits == 4'd15);
assign hex0[3] = (first_bits == 4'd1 || first_bits == 4'd4 || first_bits == 4'd7 || first_bits == 4'd9 || first_bits == 4'd10 || first_bits == 4'd15);
assign hex0[4] = (first_bits == 4'd1 || first_bits == 4'd3 || first_bits == 4'd4 || first_bits == 4'd5 || first_bits == 4'd7 || first_bits == 4'd9);
assign hex0[5] = (first_bits == 4'd1 || first_bits == 4'd2 || first_bits == 4'd3 || first_bits == 4'd7 || first_bits == 4'd13);
assign hex0[6] = (first_bits == 4'd0 || first_bits == 4'd1 || first_bits == 4'd7 || first_bits == 4'd12);

assign hex1[0] = (second_bits == 4'd1 || second_bits == 4'd4 || second_bits == 4'd11 || second_bits == 4'd13);
assign hex1[1] = (second_bits == 4'd5 || second_bits == 4'd6 || second_bits == 4'd11 || second_bits == 4'd14 || second_bits == 4'd15);
assign hex1[2] = (second_bits == 4'd2 || second_bits == 4'd12 || second_bits == 4'd14 || second_bits == 4'd15);
assign hex1[3] = (second_bits == 4'd1 || second_bits == 4'd4 || second_bits == 4'd7 || second_bits == 4'd9 || second_bits == 4'd10 || second_bits == 4'd15);
assign hex1[4] = (second_bits == 4'd1 || second_bits == 4'd3 || second_bits == 4'd4 || second_bits == 4'd5 || second_bits == 4'd7 || second_bits == 4'd9);
assign hex1[5] = (second_bits == 4'd1 || second_bits == 4'd2 || second_bits == 4'd3 || second_bits == 4'd7 || second_bits == 4'd13);
assign hex1[6] = (second_bits == 4'd0 || second_bits == 4'd1 || second_bits == 4'd7 || second_bits == 4'd12);
```

این ماژول کاربردی برای محاسبه نتیجه و درستی مدار ندارد و صرفاً به نمایش روی seven segment کمک می‌کند و در نتیجه آن را کامپایل نکردیم، بلکه RTL_view و ... که به پیوست ارسال شده‌اند مربوط به سنتز ماژول Cpu هستند.